Internship Program

Submitted By: Maryam Rehan

Mentor: Sir Sami Ullah

Project Lab Report

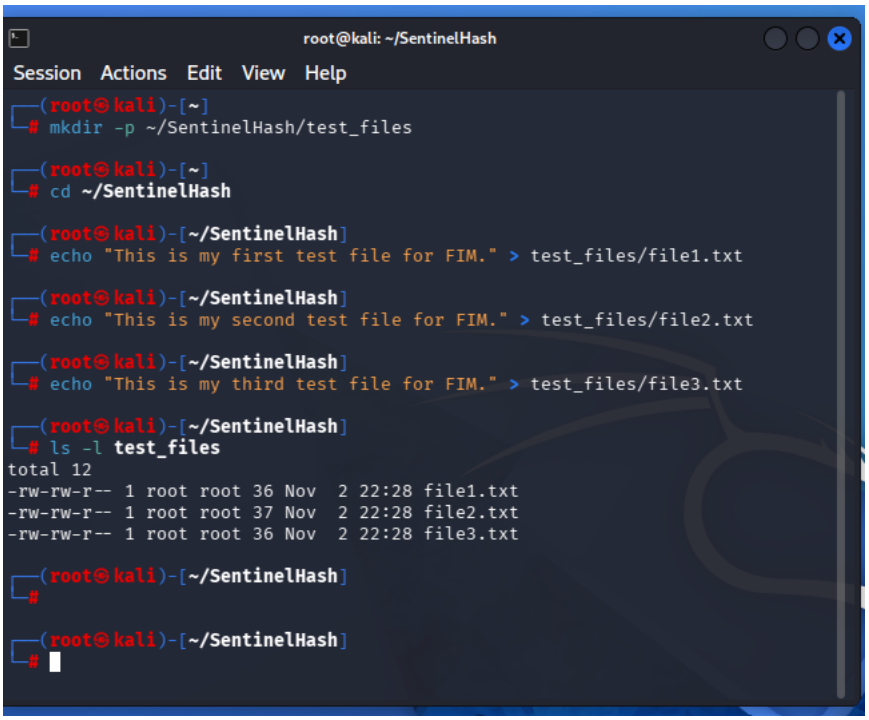
Data Integrity and Cryptography Basics

SentinelHash – File Integrity Monitoring (FIM) System Using SHA-256 Hashing

1. Objective

The objective of the SentinelHash project is to develop a basic File Integrity Monitoring (FIM) system using SHA-256 hashing. The project aims to create a trusted baseline of files, protect the baseline using AES-256 encryption, and detect unauthorized file changes, including added, modified, and deleted files. It also demonstrates how cryptographic hashing can help maintain file integrity and support basic security monitoring.

2. Proves



```
┌──(root㉿kali)-[~/SentinelHash]
└─# cd ~/SentinelHash

┌──(root㉿kali)-[~/SentinelHash]
└─# sha256sum test_files/* > baseline.txt

┌──(root㉿kali)-[~/SentinelHash]
└─# cat baseline.txt
0d64fd0aaa5882e18db89f762d78299ca3dcc6ff53815cf351d566e5b064781d  test_files/
file1.txt
1dcc27aabb9a0b9cc3909215543a92eeebbeba36acdc81c38f004c2f8a5daea1  test_files/
file2.txt
4ccead984cf6f685e9e9137e00baf92d0ba00cbdcd91c1da6a8c6aae5170907e  test_files/
file3.txt

┌──(root㉿kali)-[~/SentinelHash]
└─# sha256sum -c baseline.txt
test_files/file1.txt: OK
test_files/file2.txt: OK
test_files/file3.txt: OK

┌──(root㉿kali)-[~/SentinelHash]
└─#
```

```
┌──(root㉿kali)-[~/SentinelHash]
└─# gpg --version
gpg (GnuPG) 2.4.9
libgcrypt 1.12.2
Copyright (C) 2025 g10 Code GmbH
License GNU GPL-3.0-or-later <https://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Home: /root/.gnupg
Supported algorithms:
Pubkey: RSA, ELG, DSA, ECDH, ECDSA, EDDSA
Cipher: IDEA, 3DES, CAST5, BLOWFISH, AES, AES192, AES256, TWOFISH,
        CAMELLIA128, CAMELLIA192, CAMELLIA256
Hash: SHA1, RIPEMD160, SHA256, SHA384, SHA512, SHA224
Compression: Uncompressed, ZIP, ZLIB, BZIP2

┌──(root㉿kali)-[~/SentinelHash]
└─#
```

```
┌──(root㉿kali)-[~/SentinelHash]
└─# gpg --version
gpg (GnuPG) 2.4.9
libgcrypt 1.12.2
Copyright (C) 2025 g10 Code GmbH
License GNU GPL-3.0-or-later <https://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Home: /root/.gnupg
Supported algorithms:
Pubkey: RSA, ELG, DSA, ECDH, ECDSA, EDDSA
Cipher: IDEA, 3DES, CAST5, BLOWFISH, AES, AES192, AES256, TWOFISH,
        CAMELLIA128, CAMELLIA192, CAMELLIA256
Hash: SHA1, RIPEMD160, SHA256, SHA384, SHA512, SHA224
Compression: Uncompressed, ZIP, ZLIB, BZIP2

┌──(root㉿kali)-[~/SentinelHash]
└─# GPG_TTY=$(tty)
export GPG_TTY
gpg-connect-agent /bye
gpg-connect-agent: no running gpg-agent - starting '/usr/bin/gpg-agent'
gpg-connect-agent: waiting for the agent to come up ... (8s)
gpg-connect-agent: connection to the agent established
```

```
┌──(root㉿kali)-[~/SentinelHash]
└─# gpg --pinentry-mode loopback --symmetric --cipher-algo AES256 --output ba
seline.txt.gpg baseline.txt

┌──(root㉿kali)-[~/SentinelHash]
└─# ls -lh baseline.txt baseline.txt.gpg
-rw-rw-r-- 1 root root 261 Nov  2 22:33 baseline.txt
-rw-rw-r-- 1 root root 243 Nov  2 23:09 baseline.txt.gpg

┌──(root㉿kali)-[~/SentinelHash]
└─# gpg --pinentry-mode loopback --decrypt baseline.txt.gpg > baseline_decryp
ted.txt
gpg: AES256.CFB encrypted data
gpg: encrypted with 1 passphrase

┌──(root㉿kali)-[~/SentinelHash]
└─# cmp -s baseline.txt baseline_decrypted.txt && echo "SUCCESS: Decrypted fi
le matches original baseline."
SUCCESS: Decrypted file matches original baseline.

┌──(root㉿kali)-[~/SentinelHash]
└─#
```

```
┌──(root㉿kali)-[~/SentinelHash]
└─# cd ~/SentinelHash
nano fim_checker.py

┌──(root㉿kali)-[~/SentinelHash]
└─# python3 -m py_compile fim_checker.py

┌──(root㉿kali)-[~/SentinelHash]
└─# python3 fim_checker.py
[OK] No changes detected. All files match the baseline.

┌──(root㉿kali)-[~/SentinelHash]
└─# echo "This is a newly added test file." > test_files/file4.txt

┌──(root㉿kali)-[~/SentinelHash]
└─# date
python3 fim_checker.py
Mon Nov  2 11:16:25 PM EST 2026
[ADDED] test_files/file4.txt
```

```
┌──(root㉿kali)-[~/SentinelHash]
└─# rm test_files/file4.txt

┌──(root㉿kali)-[~/SentinelHash]
└─#
```

```
┌──(root㉿kali)-[~/SentinelHash]
└─# echo "This content has been modified for FIM testing." >> test_files/file
1.txt

┌──(root㉿kali)-[~/SentinelHash]
└─# date
python3 fim_checker.py
Mon Nov  2 11:19:19 PM EST 2026
[MODIFIED] test_files/file1.txt

┌──(root㉿kali)-[~/SentinelHash]
└─#
```

```
┌──(root㉿kali)-[~/SentinelHash]
└─# printf '%s\n' "This is my first test file for FIM." > test_files/file1.tx
t

┌──(root㉿kali)-[~/SentinelHash]
└─#
```

```
┌──(root㉿kali)-[~/SentinelHash]
└─# rm test_files/file3.txt

┌──(root㉿kali)-[~/SentinelHash]
└─# date
python3 fim_checker.py
Mon Nov  2 11:22:07 PM EST 2026
[DELETED] test_files/file3.txt

┌──(root㉿kali)-[~/SentinelHash]
└─# sha256sum -c baseline.txt
test_files/file1.txt: OK
test_files/file2.txt: OK
sha256sum: test_files/file3.txt: No such file or directory
test_files/file3.txt: FAILED open or read
sha256sum: WARNING: 1 listed file could not be read

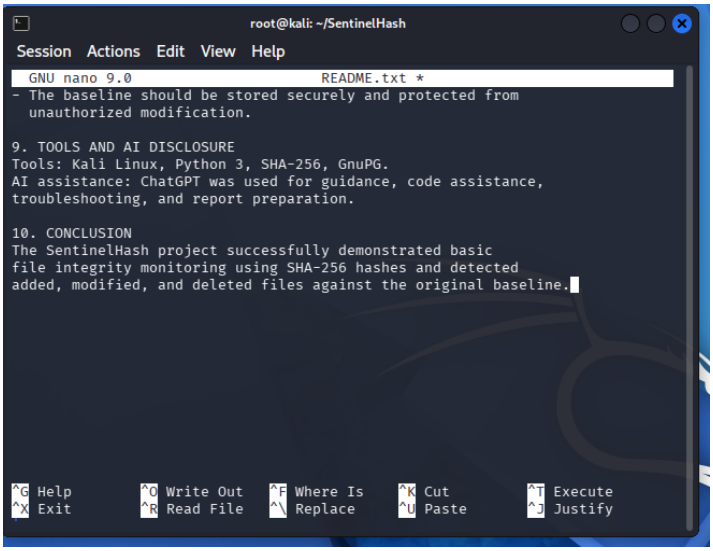
┌──(root㉿kali)-[~/SentinelHash]
└─#
```

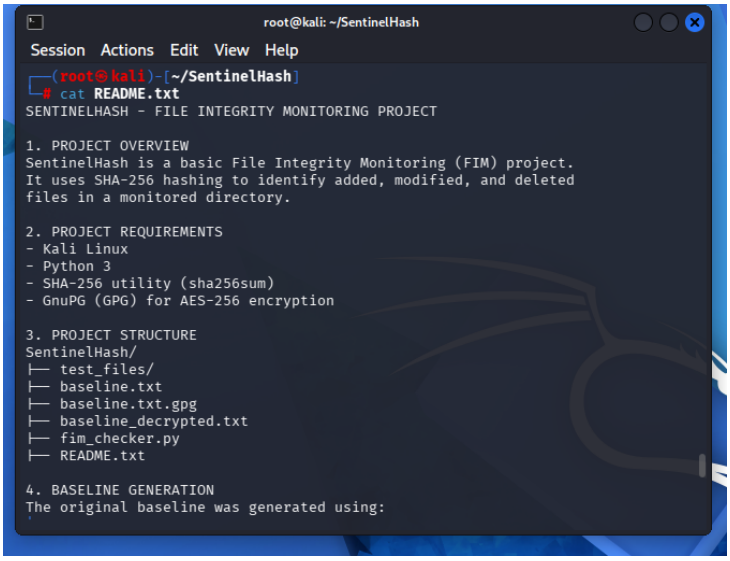
```
┌──(root㉿kali)-[~/SentinelHash]
└─# ls -lh
total 24K
-rw-rw-r-- 1 root root  261 Nov  2 23:11 baseline_decrypted.txt
-rw-rw-r-- 1 root root  261 Nov  2 22:33 baseline.txt
-rw-rw-r-- 1 root root  243 Nov  2 23:09 baseline.txt.gpg
-rw-rw-r-- 1 root root 1.3K Nov  2 23:14 fim_checker.py
drwxrwxr-x 2 root root 4.0K Nov  2 23:15 __pycache__
drwxrwxr-x 2 root root 4.0K Nov  2 23:21 test_files

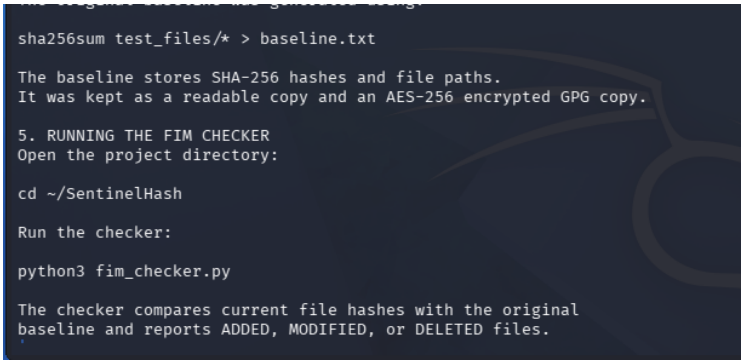
┌──(root㉿kali)-[~/SentinelHash]
└─# ls -lh test_files/
total 8.0K
-rw-rw-r-- 1 root root 36 Nov  2 23:20 file1.txt
-rw-rw-r-- 1 root root 37 Nov  2 22:28 file2.txt

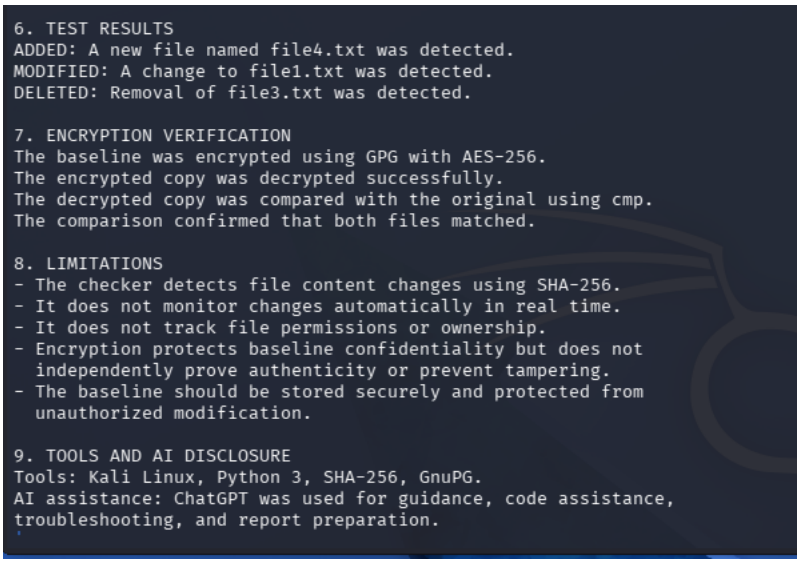
┌──(root㉿kali)-[~/SentinelHash]
└─# rm -r __pycache__

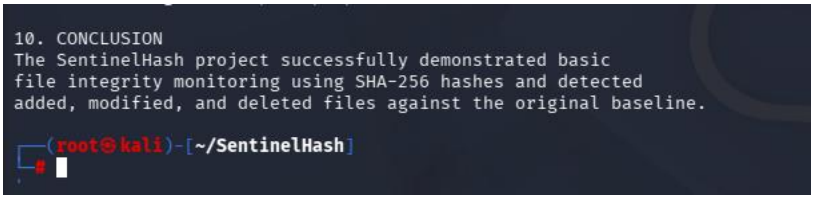
┌──(root㉿kali)-[~/SentinelHash]
└─#
```




root@kali: ~/SentinelHash
Session Actions Edit View Help
┌──(root㉿kali)-[~/SentinelHash]
└─# cat README.txt
SENTINELHASH - FILE INTEGRITY MONITORING PROJECT
1. PROJECT OVERVIEW
SentinelHash is a basic File Integrity Monitoring (FIM) project.
It uses SHA-256 hashing to identify added, modified, and deleted
files in a monitored directory.
2. PROJECT REQUIREMENTS
- Kali Linux
- Python 3
- SHA-256 utility (sha256sum)
- GnuPG (GPG) for AES-256 encryption
3. PROJECT STRUCTURE
SentinelHash/
├── test_files/
├── baseline.txt
├── baseline.txt.gpg
├── baseline_decrypted.txt
├── fim_checker.py
├── README.txt
4. BASELINE GENERATION
The original baseline was generated using:


sha256sum test_files/* > baseline.txt
The baseline stores SHA-256 hashes and file paths.
It was kept as a readable copy and an AES-256 encrypted GPG copy.
5. RUNNING THE FIM CHECKER
Open the project directory:
cd ~/SentinelHash
Run the checker:
python3 fim_checker.py
The checker compares current file hashes with the original
baseline and reports ADDED, MODIFIED, or DELETED files.


6. TEST RESULTS
ADDED: A new file named file4.txt was detected.
MODIFIED: A change to file1.txt was detected.
DELETED: Removal of file3.txt was detected.
7. ENCRYPTION VERIFICATION
The baseline was encrypted using GPG with AES-256.
The encrypted copy was decrypted successfully.
The decrypted copy was compared with the original using cmp.
The comparison confirmed that both files matched.
8. LIMITATIONS
- The checker detects file content changes using SHA-256.
- It does not monitor changes automatically in real time.
- It does not track file permissions or ownership.
- Encryption protects baseline confidentiality but does not
independently prove authenticity or prevent tampering.
- The baseline should be stored securely and protected from
unauthorized modification.
9. TOOLS AND AI DISCLOSURE
Tools: Kali Linux, Python 3, SHA-256, GnuPG.
AI assistance: ChatGPT was used for guidance, code assistance,
troubleshooting, and report preparation.



3.Working / Methodology

SentinelHash works by generating SHA-256 hashes for three sample files stored in a monitored directory. These hashes are saved in a baseline file named baseline.txt.

The baseline is also encrypted using GPG with AES-256 encryption to protect its confidentiality. The encrypted file is decrypted successfully and compared with the original baseline to verify its accuracy.

A Python script named fim_checker.py compares the current file hashes with the original baseline. It identifies three types of changes:

- ADDED: Detects newly created files that are not included in the baseline.
- MODIFIED: Detects changes in existing files by comparing their SHA-256 hashes.
- DELETED: Detects files that existed in the original baseline but are now missing.

The project successfully detected all three types of changes during practical testing.

4. Conclusion

The SentinelHash project successfully demonstrated basic File Integrity Monitoring using SHA-256 hashing and Python. It generated and protected a file baseline using AES-256 encryption and verified successful decryption. The FIM checker accurately detected added, modified, and deleted files during testing. This project improved practical understanding of file integrity, cryptographic hashing, encryption, and basic security monitoring. It also highlighted the importance of protecting baseline files against unauthorized modification.

Basic Security Concepts

1. Integrity

Integrity means ensuring that data remains accurate, complete, and unchanged unless an authorized modification is made. It helps protect information from unauthorized changes.

2. Hashing

Hashing is a one-way mathematical process that converts data into a fixed-length hash value. It is used to verify file integrity and detect unauthorized changes.

3. Encryption

Encryption converts readable information into unreadable ciphertext using an encryption key. It protects sensitive information and maintains confidentiality.

4. Digital Signatures

Digital signatures use cryptographic keys to verify the authenticity and integrity of digital messages or files. They help confirm that information comes from a trusted sender and has not been altered.

Part B: Comparison of Hashing Algorithms

MD5

MD5 generates a 128-bit hash value. It is considered weak because attackers can create different files with the same hash due to known collision vulnerabilities.

SHA-1

SHA-1 generates a 160-bit hash value. It is also considered insecure for modern security applications because practical collision attacks have been demonstrated.

SHA-256

SHA-256 generates a 256-bit hash value. It provides stronger security and collision resistance compared with MD5 and SHA-1.

My Choice: SHA-256

I would choose SHA-256 for a new file integrity monitoring system because it provides stronger security and better collision resistance. SentinelHash uses SHA-256 to calculate and compare file hashes.

Part E: SOC Analyst Finding and Improvement

SOC Analyst Finding

During the SentinelHash investigation, the File Integrity Monitoring system successfully detected three types of file changes. A newly created file was identified as ADDED, an existing file modification was detected as MODIFIED, and a removed file was reported as DELETED.

These changes may indicate unauthorized file activity. A SOC analyst should investigate the events to determine whether they were caused by legitimate administrative actions or potentially malicious activity.

Recommended Improvement

The system can be improved by integrating FIM alerts with system authentication logs and process execution logs. This would help SOC analysts identify which user or process caused a file change and distinguish legitimate activity from suspicious behavior.

Knowledge Check – Review Questions and Answers

Q1. Does hashing encrypt a file?

Answer: No. Hashing is a one-way mathematical process used to verify data integrity. Encryption protects confidentiality and can be reversed using the correct decryption key.

Q2. Why is SHA-256 preferred over MD5 for a new integrity check?

Answer: SHA-256 provides stronger collision resistance, while MD5 has known collision vulnerabilities. Therefore, SHA-256 is a better choice for modern integrity monitoring.

Q3. What is a baseline in File Integrity Monitoring?

Answer: A baseline is a trusted record of the original state of files, including their hashes and attributes. It is used to compare future file states and detect changes.

Q4. Name three FIM change types.

Answer: The three main FIM change types are Added, Modified, and Deleted. File permission and attribute changes can also be monitored.

Q5. Why should a SOC correlate FIM with logon and process events?

Answer: Correlating FIM alerts with logon and process events helps identify which user or process caused a file change. This context helps SOC analysts distinguish authorized administrative activity from malicious file tampering.